

Студент: Гринько Мария

Группа: РПО-3 2025

Дисциплина: Архитектура аппаратных средств

Биты

Всё, с чем работает компьютер, хранится в виде битов. Каждый бит — это либо 0, либо 1.

Бит — это минимальная единица информации. Других значений у него нет. Так сделано потому, что электронные элементы легко хранят два состояния. Есть напряжение — это единица. Нет напряжения — это ноль.

Почему именно два состояния

Два состояния надёжнее, чем десять или сто. Даже если сигнал исказится помехами, устройство всё равно сможет понять, был это ноль или единица. Если бы использовалось много уровней напряжения, любая помеха приводила бы к ошибке. А с двумя состояниями запас надёжности большой.

Именно поэтому вся вычислительная техника построена на двоичной системе.

Байт

Восемь битов образуют байт. Байт вмещает числа от 0 до 255.

Этого хватает для кодирования одного символа текста или небольшого числа. Байт — это основная единица измерения объёма данных.

Когда говорят «размер файла 1 килобайт», имеют в виду 1024 байта. Это примерно тысяча символов текста.

Шестнадцатеричная система

Для удобства записи больших чисел используют шестнадцатеричную систему. В ней 16 цифр: от 0 до 9 и от A до F.

Одна шестнадцатеричная цифра заменяет ровно четыре бита. Это очень удобно: байт — это две шестнадцатеричные цифры.

Например, число 15213 в двоичном виде — 0011101101101101. В шестнадцатеричном — 3B6D.

Запись чисел в Си

В языке Си шестнадцатеричные числа записывают с префиксом 0x.

Например, 0x3B6D.

Двоичные числа в Си записать напрямую нельзя. Их обычно записывают в шестнадцатеричном виде, потому что это короче и понятнее.

Компилятор всё равно переводит числа в двоичный вид.

Шестнадцатеричная запись нужна только для удобства человека.

Машинные слова

Процессор работает не с отдельными битами, а со словами. Слово — это блок фиксированного размера.

Размер слова определяет, сколько данных процессор обрабатывает за одну операцию. Это важная характеристика архитектуры.

На 32-битных машинах слово равно 4 байтам. На 64-битных — 8 байтам.

Адресация слов

Адрес слова — это адрес его первого байта. Все остальные байты слова идут следом.

Адреса соседних слов различаются на 4 или на 8 — в зависимости от разрядности. Это значит, что адреса выровнены.

Например, на 32-битной машине слова могут начинаться с адресов 0, 4, 8, 12 и так далее.

Порядок байтов

Байты внутри слова можно располагать по-разному. Есть два основных порядка.

На процессорах x86 младший байт идет первым. Такой порядок называют остроконечным, или little-endian.

На процессорах Sun и в интернете старший байт идет первым. Это тупоконечный порядок, или big-endian.

Зачем нужен порядок байтов

Разница важна при передаче данных между системами. Если не учитывать порядок, числа будут интерпретироваться неправильно.

Например, число 0x01234567 на x86 будет лежать как 67 45 23 01. На Sun — как 01 23 45 67.

При передаче файла или сетевого пакета нужно явно указывать порядок байтов.

Булева алгебра

Логические операции придумал Джордж Буль в XIX веке. В его алгебре истина — это 1, а ложь — это 0.

Буль хотел описать законы человеческого мышления математически. Но его работа оказалась гораздо полезнее для техники.

Сегодня булева алгебра — основа всех цифровых схем.

Операция И

Операция И даёт 1 только тогда, когда оба входа равны 1. В остальных случаях результат 0.

Это как два выключателя, соединенных последовательно. Свет загорится только если оба включены.

В языке Си операция И обозначается символом $\&$.

Операция ИЛИ

Операция ИЛИ даёт 1, если хотя бы один вход равен 1. Если оба входа 0, результат тоже 0.

Это как два выключателя, соединенных параллельно. Свет загорится, если включен хотя бы один.

В языке Си операция ИЛИ обозначается символом $|$.

Операция НЕ

Операция НЕ инвертирует значение. 0 превращается в 1, а 1 в 0.

Это как выключатель с перекидным контактом. Одно положение даёт одно значение, другое — противоположное.

В языке Си операция НЕ обозначается символом $\sim$.

Исключающее ИЛИ

Исключающее ИЛИ даёт 1, если входы разные. Если оба входа одинаковые, результат 0.

Это удобно для проверки различий. Если два бита совпадают, результат 0, если различаются — 1.

В языке Си операция обозначается символом $\wedge$.

Применение логики к схемам

Клод Шеннон в 1937 году доказал, что булеву алгебру можно применять к электрическим схемам.

Замкнутый контакт — это 1, разомкнутый — 0. Из таких контактов можно собрать любую логическую схему.

Эта работа Шеннона стала основой для построения процессоров.

Побитовые операции в Си

В языке Си есть побитовые операции: $\&$, $|$, $\sim$, $\wedge$. Они работают с каждым битом числа независимо.

Например, $0x69 \& 0x55$ даёт $0x41$. Каждый бит первого числа сравнивается с соответствующим битом второго.

Побитовые операции применяются к целым типам: `char`, `short`, `int`, `long`.

Логические операции в Си

Есть также логические операции: $\&\&$, $||$, $!$. Они работают со значением целиком.

Логические операции всегда возвращают 0 или 1. Ноль считается ложью, всё остальное — истиной.

Например, $0x69 \&\& 0x55$ даёт 1, потому что оба числа не равны нулю.

Разница между побитовыми и логическими

Важно не путать эти два вида операций. Побитовые работают с битами, логические — с истиной и ложью.

$0x69 \& 0x55$ — это $0x41$. А $0x69 \&\& 0x55$ — это 1. Результаты совершенно разные.

Ошибка в выборе операции — частая проблема начинающих программистов.

Отрицательные числа

Отрицательные числа хранятся в дополнительном коде. Идея простая: $x + (-x) = 0$.

Из этого следует, что $-x = \sim x + 1$. То есть нужно инвертировать все биты и прибавить единицу.

Такой способ позволяет складывать положительные и отрицательные числа одинаково.

Знаковый бит

Старший бит обозначает знак: 0 — число положительное, 1 — отрицательное.

Остальные биты хранят само значение. Но интерпретация зависит от того, знаковое число или беззнаковое.

Одно и то же сочетание битов может означать разные числа.

Пример дополнительного кода

Число -15213 в 16-битном представлении выглядит как 1100010010010011.

Если интерпретировать это как беззнаковое число, получится 50123.

Если как знаковое — получится -15213. Всё зависит от типа.

Диапазон знаковых чисел

Диапазон знаковых чисел несимметричен. Минимальное число по модулю на единицу больше максимального.

Для 16 бит диапазон — от -32768 до 32767. Для 32 бит — примерно от -2 миллиардов до 2 миллиардов.

Это следствие того, что ноль занимает одно значение.

Почему диапазон несимметричен

Положительных чисел столько же, сколько отрицательных, но ноль один.

Поэтому отрицательных чисел на одно больше. Минимальное отрицательное число не имеет положительного аналога.

Это важно помнить при работе с отрицательными числами.

Переполнение

Когда результат сложения выходит за границы диапазона, возникает переполнение.

Процессор не считает это ошибкой. Он просто отбрасывает лишний бит и продолжает работу.

Программист должен сам следить за переполнением, если оно важно.

Беззнаковое переполнение

При беззнаковом сложении результат «заворачивается» по модулю 2^w в степени w .

Например, $255 + 1$ даёт 0, если число однобайтовое. Единица переноса отбрасывается.

Это похоже на счетчик, который после максимума сбрасывается в ноль.

Знаковое переполнение

При знаковом сложении переполнение может быть положительным или отрицательным.

Если складываются два положительных числа и получается отрицательное — это положительное переполнение.

Если складываются два отрицательных и получается положительное — это отрицательное переполнение.

Устройство процессора

Процессор состоит из нескольких ключевых блоков. Каждый блок выполняет свою функцию.

Дешифратор команд читает код операции и определяет, что нужно делать.

Устройство управления руководит всеми остальными блоками и вырабатывает управляющие сигналы.

АЛУ

АЛУ выполняет арифметические и логические операции. Это комбинационная схема.

Комбинационная схема не хранит данные внутри себя. Она только вычисляет результат по входным сигналам.

На вход АЛУ подаются два операнда. На выходе получается результат операции.

Регистры

Регистры хранят слова данных. Одни регистры видны программисту явно, другие используются внутри процессора.

Регистр — это линейка триггеров. Триггер хранит один бит.

Регистры могут быть программно-видимыми, теневыми или внутренними.

Шины

Шины связывают все блоки между собой. Шины бывают внутренние и внешние.

Внутренние шины соединяют блоки внутри процессора. Внешние — процессор с памятью и другими устройствами.

По шине передаются адреса, данные и управляющие сигналы.

Устройство управления

Устройство управления может быть построено двумя способами. Первый — жёсткая логика.

При жёсткой логике сигналы управления формируются однажды соединёнными логическими схемами.

Счётчик тактов сбрасывается после каждой команды. На каждом такте вырабатывается своя микрокоманда.

Программируемая логика

Второй способ — программируемая логика. Идею предложил Морис Уилкс в 1951 году.

При программируемой логике между командами и сигналами управления появляется микропрограмма.

Она хранится в специальной памяти. Каждой команде соответствует своя микропрограмма.

Микропрограммирование

Микропрограммирование позволяет упростить аппаратную часть процессора.

Но в современных процессорах от этой идеи отказались. Она стала сдерживать рост производительности.

Аппаратные затраты перестали быть проблемой, поэтому вернулись к жесткой логике.

Архитектуры системы команд

Операнды могут храниться по-разному. От этого зависит архитектура системы команд.

Есть четыре основных типа: стековая, аккумуляторная, регистровая и с выделенным доступом к памяти.

Каждая архитектура имеет свои преимущества и недостатки.

Стековая архитектура

В стековой архитектуре данные лежат в стеке по принципу «последним вошел — первым вышел».

Вычисления записываются в обратной польской записи. Это позволяет обходиться без скобок.

Стек бывает программный и аппаратный.

Аккумуляторная архитектура

В аккумуляторной архитектуре один из операндов всегда находится в аккумуляторе.

Второй операнд — в памяти. Команды короткие, но приходится часто обращаться к памяти.

Это замедляет работу, зато упрощает декодирование команд.

Регистровая архитектура

В регистровой архитектуре есть массив регистров общего назначения.

Операнды могут быть в регистрах или в памяти. Команды обработки делятся на три подвида.

Это дает гибкость, но требует более длинных инструкций.

Архитектура с выделенным доступом к памяти

В RISC-процессорах обращение к памяти возможно только командами `load` и `store`.

Все операции выполняются над регистрами. Команды обычно имеют длину 32 бита и трехадресный формат.

Примеры таких процессоров: Sun SPARC, MIPS, DEC Alpha, PowerPC.

Процессор Intel 8086

Intel 8086 — классический пример CISC-архитектуры. У него больше 500 команд.

Длина команды может быть от 1 до 15 байтов. Форматы кодирования сложные.

Много специализированных регистров. Структура не вполне ортогональна.

Сегментная адресация

Память организована сегментно. Адрес формируется из сегментного регистра и смещения.

Физический адрес получается умножением сегмента на 16 и прибавлением смещения.

Это позволяет адресовать больше памяти, чем позволяет разрядность регистров.

Рабочий цикл процессора

Выполнение команды состоит из нескольких этапов. Сначала процессор выставляет на шину адреса значение из счётчика команд.

Команда считывается из памяти в регистр команд. Потом из регистра команд на шину адреса выставляется адрес первого операнда.

Первый операнд считывается в аккумулятор. Затем считывается второй операнд во второй регистр АЛУ.

Этапы выполнения

АЛУ выполняет операцию. Результат попадает в аккумулятор.

После этого результат записывается обратно в память на место первого операнда.

Счетчик команд увеличивается на длину выполненной команды.

Переход к следующей команде

Так происходит переход к следующей команде. Цикл повторяется.

Счетчик команд всегда указывает на следующую команду. Это обеспечивает последовательное выполнение программы.

Для перехода к произвольной команде нужны команды передачи управления.

Прерывания

Во время работы программы могут возникать прерывания. Это исключения вроде деления на ноль или переполнения.

Другой тип — запросы от внешних устройств. Например, нажатие клавиши или готовность к вводу-выводу.

Прерывания позволяют процессору реагировать на события в реальном времени.

Поддержка прерываний

Для поддержки прерываний в процессоре есть регистр флагов и регистр-указатель стека.

При возникновении прерывания адрес возврата сохраняется в стек. Затем процессор переходит на программу обработки прерывания.

Таблица векторов прерываний

Адрес программы берется из таблицы векторов прерываний. Каждому типу прерывания соответствует своя ячейка.

В ячейке записана команда безусловного перехода на адрес первой команды обработчика.

Эта таблица хранится в начале оперативной памяти.

Возврат из прерывания

Последняя команда обработчика — RETI. Она возвращает управление прерванной программе.

Из стека извлекается сохранённый адрес возврата и загружается в счетчик команд.

Прерывания от внешних устройств можно замаскировать. Исключения, как правило, замаскировать нельзя.

Мультипрограммный режим

Современные компьютеры работают в мультипрограммном режиме.

Каждой программе выделяется квант времени.

По истечении кванта возникает прерывание от таймера. Процессор переключается на другую программу.

Поскольку квант мал, у пользователя создаётся иллюзия непрерывной работы.

Маскирование прерываний

Для маскирования прерываний в процессоре есть регистр масок. Каждый бит соответствует причине прерывания.

Если бит установлен в единицу, прерывание не обрабатывается. Это позволяет временно отключить реакцию на некоторые события.

Не все прерывания можно замаскировать. Исключения обычно обрабатываются всегда.